Efficient Weight Mapping and Resource Scheduling on Crossbar-based Multi-core CIM Systems

Hanjie Liu†, Sifan Sun†, Aifei Zhang‡, Haiyan Qin†, Yutong Wu‡, Minhao Gu‡,
Shihang Fu‡, Shuaikai Liu‡, Baosen Liu‡, Wang Kang*†
†*National Key Laboratory of Spintronics, Hangzhou International Innovation Institute;*
School of Integrated Circuit Science and Engineering, Beihang University, China
‡*Zhicun Research Lab, Beijing, China*
**Corresponding author: wang.kang@buaa.edu.cn*

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132743

***Abstract*—Crossbar-based computing-in-memory (CIM) systems facilitate large-scale parallel multiply-and-accumulate (MAC) operations, while a domain-specific compiler (DSC) plays a pivotal role in optimizing the deployment of neural network algorithms on such systems. With the development of multi-core and large-core architectures, some key compiler problems such as high parallel processing, resource utilization, and crossbar array assignment methods have not been solved. For low-latency application scenarios, we have designed a resource scheduling strategy for our hardware system based on stream data processing to reduce the latency caused by intra-core and inter-core communication. Additionally, a weight mapping strategy has been developed to maximize the potential of crossbar arrays in convolutional neural networks (CNNs) deployment. Experimental results on our multi-core eFlash-based CIM system-on-chip (SoC) demonstrate that these two technologies help CNNs achieve a 76% reduction in latency, a 30% improvement in resource utilization, and the use rate of crossbar array that can reach up to 94.7%.**

***Index Terms*—CIM, CNN, Compiler, Weight mapping, Resource schedule**

I. Introduction

In recent years, the extensive application of convolutional neural networks (CNNs) in machine learning has generated a substantial demand for efficient architectures to manage these computationally and data-intensive tasks. In response, various neural network accelerators [1], [2] have emerged. However, these CMOS-based devices, which adhere to the von Neumann architecture, are now facing the challenge posed by "the memory wall" [3].

Computing-in-memory (CIM) technology provides a promising solution for accelerating artificial intelligence inference by performing analog computations directly in memory, potentially reducing latency and power consumption. To effectively implement CIM, various techniques have been proposed, including commercial memory types such as SRAM, DRAM, and flash memory, as well as emerging non-volatile memory (NVM) technologies like ReRAM, PCM, FeFET, and MRAM [4]. The two-dimensional crossbar arrays constructed from these devices exhibit high storage density and the capability for parallel in-situ computation, thus garnering significant academic interest.

The development of macro designs for crossbar-based CIM accelerators has advanced rapidly, yet there remains a lack of detailed consideration for the execution of CNNs, necessitating manual intervention for actual deployment. As task complexity increases and neural networks continue to deepen, the demand for computational power also grows. Consequently, macro designs are evolving towards multi-core and large-core architectures, and there is an increasing call for automated deployment solutions.

Currently, the automation of compilation in some CIM systems focuses on small-core implementation strategies [5], [6]. These strategies enhance parallelism by partitioning convolution weights and distributing them across different cores for computation. However, this splitting introduces the need for summing and concatenating multi-core output results, which can accumulate significant quantization errors during actual computations [7]. While large-core computation eliminates these issues, it presents new challenges in optimizing resource utilization and data flow.

In our work, we achieve automated optimization and deploy CNNs on a multi-core CIM system, resulting in efficient, low-latency, and highly parallel task execution. Building on multi-core collaboration and large crossbar arrays, we propose two technical improvements for resource utilization. Our specific contributions are as follows:

- A multi-level abstract architecture and execution logic for a multi-core CIM system is proposed, serving as the foundation for detailed research on CNNs execution.
- A scheduling process is designed to improve system computational resource utilization and optimize latency caused by bandwidth limitations through resource pooling, static random allocation, and iterative evaluation.
- A new weight mapping strategy is designed for lager-core, which leverages weighted expansion to exploit the potential of crossbar arrays in processing convolutions.
- Experimental results on our multi-core CIM system-on-chip (SoC) demonstrate a 76% reduction in latency, a 30% improvement in resource utilization, and a crossbar array utilization rate of up to 94.7%.

II. Background and Motivation

Fig. 1 illustrates the basic operational unit for analog vector-matrix multiplication (VMM) in CIM systems, implemented within a two-dimensional crossbar array structure. This computational strategy can be categorized into two types based on the physical properties of the devices [8].

This work was supported by the Beijing MSTC Program (Z231100007423019), Beijing Natural Science Foundation (L223004), Natural Science Foundation of China (62274008).

A. Crossbar-based CIM

Fig. 1(a) shows the floating-gate memory cells array. In the subthreshold region, the leakage current of floating-gate devices exhibits an exponential dependence on the gate voltage. At the same time, the gate voltage difference can represent the ratio of input to output current. Therefore, we can program the device by adjusting the charge at the cell intersections and the peripheral unit charge, with the output current component expressed as: $I_{\Sigma i} = W_i I_i$. In the NVM crossbar structure shown in Fig. 1(b), the conductance value of each cell is programmable, and the current at each intersection is given by: $I_{\Sigma i} = G_i V_i$. According to Kirchhoff's law, the cumulative current value in each column is the sum of its components. Thus, the crossbar array completes the VMM operation.

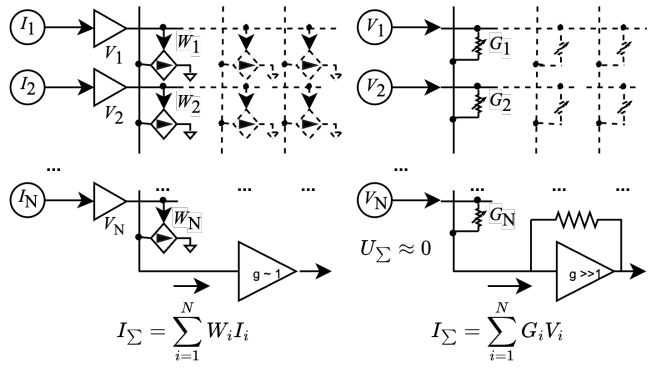

Fig. 1. Analog VMM circuits with Floating-gate crossbar and NVM crossbar. Note the difference in Op-Amp gain requirements.

NVM devices have fast write speeds and high write endurance, showcasing strong development potential. In contrast, floating-gate devices boast high density and mature processes, making them more advantageous in large-core designs [9]. Given that our SoC is built on floating-gate technology, our approach focuses on studying CIM systems based on large-scale floating-gate arrays.

B. Auto Development for CNNs

To achieve automated deployment of CNNs in CIM systems, industry and academia [10], [11] have introduced domain-specific compilers (DSC). These compilers typically consist of frontend and backend. The frontend is responsible for hardware-agnostic optimizations, including computational graph and operator optimization, often converting other operators into matrix computation operators such as convolution and fully connected. The backend emphasizes optimization strategies for the storage and computation of matrices involved in matrix-vector multiplication (MVM) as shown in Fig. 2.

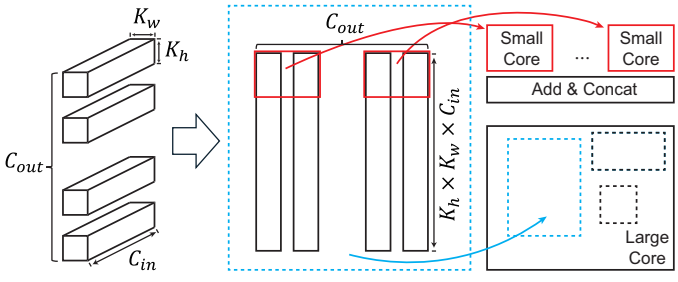

Fig. 2. Weight mapping of CNNs on different size of crossbar. K_w and K_h represent the width and height of convolution kernel, C_{in} and C_{out} represent the number of input and output channels. Weights are flattened and storage in the crossbar.

In systems based on NVM, to perform computations of larger-scale weight matrices within the limited size of crossbar arrays. Studies [5], [11] have adopted methods that split the original weights and distribute them across multiple cores for computation. This approach addresses the problem and enhances computational parallelism, Nevertheless, reorganizing the computation results introduces additional computational overhead and quantization errors.

In floating-gate and future NVM-based systems, when the weight matrix is smaller than the crossbar array, the entire matrix can be stored in a single core, thus eliminating the need for splitting computations and subsequent summation and stitching operations. However, maintaining high parallelism becomes a new challenge. Additionally, with the increase in single-core computational tasks, the optimization space for data flow both within and between cores also expands.

III. Architecture of CIM System

We propose an abstract architecture for a multi-core CIM system, which is compatible with widely adopted multi-level structures [12]. Based on this architecture, we developed an inference execution model tailored for low-latency application scenarios, enabling comprehensive analysis and evaluation of optimization strategies in the compilation backend.

A. Hardware Abstraction

Fig. 3 illustrates the proposed multi-level architecture, which sequentially expands from the Chip level to the Core, Matrix Computing Unit (MCU) and Crossbar levels.

At the highest level, the architecture consists of a two-dimensional homogeneous array of computing modules and a global buffer, where each computing module includes a CIM core and an inter-buffer. Modules are interconnected through a Network-on-Chip (NoC). The CNN parameters are pre-stored in the cores, and data exchange between cores occurs through the inter-buffer and global buffer. Different cores execute operations asynchronously, using a trigger-off mechanism for synchronization during inter-core communication.

Each core comprises a control unit, an inner-buffer, a set of MCUs, and other computation units (OCUs). These OCUs include element-wise operations (e.g., batch normalization), vector operations (e.g., pooling), and transformation operations (e.g., sampling). The computation units within the core can only access the data stored in the inner-buffer.

Each MCU consists of input/output first-in-first-out (FIFO) queues, pre-processing and post-processing units, and a set of crossbar arrays. The pre- and post-units perform operations such as shifting, clipping, expansion, and activation to support

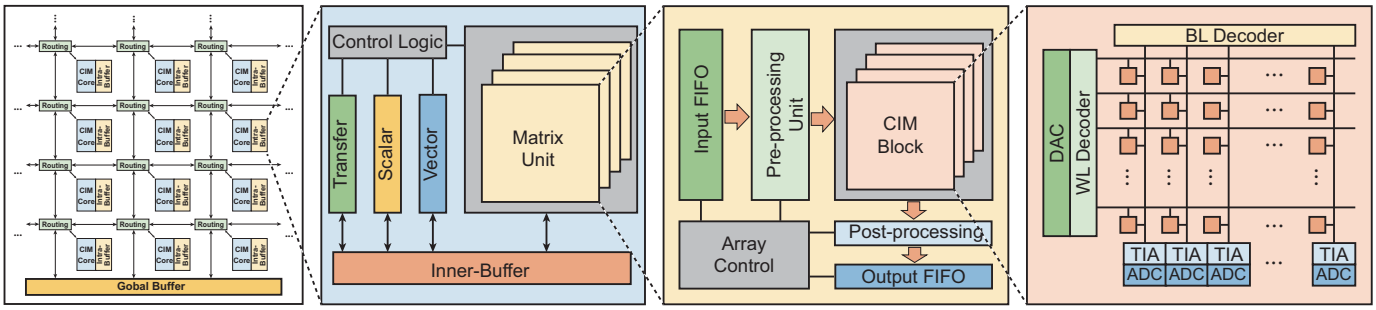

Fig. 3. Multilevel architecture of CIM system. Multiple cores interconneted through NoC in the top level. Each core contains various computing units, with MCU featuring multiple crossbars that share a common pre- and post-processing system. (a) Chip level. (b) Core level. (c) MCU level. (d) Crossbar level.

the fusion and acceleration of common operations before and after on convolution layers in CNNs.

Since the crossbar operates in the analog domain while the other circuits in the system function in the digital domain, basic peripheral circuits such as ADC/DAC and transimpedance amplifiers (TIA) are also required in the CIM Block.

B. Inference Execution

Algorithmic network tasks are divided into multiple chip pipelines for execution. Each pipeline covers a subnet, with its outputs stored in the global buffer, serving as the input for the subsequent pipelines.

Each chip pipeline is further divided into tasks, completed either within a single core or collaboratively across multiple cores. The outputs of each task, along with intermediate results across cores, are written into the inter-buffer.

Inner-core computations interact through a bank of inner-buffer. Each computation unit is allocated specific input and output spaces, facilitating parallel computation. Once a unit's designated input data is exhausted or its output space is filled, it must wait for the dependent units to process these data.

IV. Methodology

Given that the frequency of digital modules is typically much higher than that of crossbar arrays, we consider the computation time of the crossbar arrays T_{MVM} and the transfer of intermediate data T_{DT} across multiple levels as the primary components of latency T:

$$T = T_{DT} + T_{MVM} = \sum_i \frac{\text{Data}_{L_i}}{\text{Band}_{L_i}} + \sum_j \frac{F_{out_j}}{C_{out_j}} \quad (1)$$

$Data_{L_i}$ and $Band_{L_i}$ represent the data volume and bandwidth at each level, respectively. F_{out_j} and C_{out_j} represent the volume of output feature map and the output channel of the corresponding MVM layer, respectively. To reduce latency in the proposed architecture, two primary steps are involved:

- Optimize data transfer latency by allocating computational resources and intermediate data storage resources through resource scheduling.
- Optimize crossbar latency by using a weighted expansion strategy for weight mapping, which enhances resource allocation on individual crossbar arrays.

A. Resource Scheduling

Algorithm 1 outlines the overall process of scheduling. Initially, a resource management pool based on a multi-level architecture is instantiated according to the configuration parameters. Next, the computation graph of the CNN to be deployed is loaded and partitioned into multiple subgraphs. Computational and storage resources are then allocated to the operators in each layer in the subgraphs, generating the complete data flow. Finally, iterative optimization is performed on the data flow to minimize latency.

Algorithm 1 Resource Scheduling Algorithm

Require: ArchParams, CompGraph, EvalFunc
Ensure: Optimal resource allocation and data flow

```
Update resource pool by ArchParams //Resource Statistics
for each node in CompGraph do
    if random_probability < P then
        Create subgraph from current node
        for each sub_node in subgraph do
            Randomly allocate func to sub_node
        end for  //Resource Allocation
    else Update(P)
    end if
end for  //Network Partition
for each operator pair in CompGraph do
    buffer set: (diff subgraph) ? L0 : (diff core) ? L1 : L2
    buffer allocate
end for  //Data Flow Generation
for each generation do
    latency = EvalFunc(data_flow)
    Select best solutions based on latency
    Apply crossover to create new population
end for  //Evaluation Iteration
Return best solution found
```

a) Network Partition: To achieve a larger solution search space, we design the computation graph partitioning and resource allocation process to be highly stochastic. During the graph partitioning process, we determine whether to cut at the current node and generate a subgraph with a probability P,

which is expressed as:

$$P = \frac{1}{1 + e^{k(R-R_0)}}, \quad 0 \leq R \leq 1 \tag{2}$$

Where R represents the remaining resource rate. k is the scaling factor and R_0 is the threshold parameter.

b) Resource Allocation and Dataflow Generation: For computational resources, nodes' attributes are matched with the corresponding functional computation units in the resource pool without restricting the allocation to specific cores. For storage resources, only sizes are allocated, not specific addresses. The inner-buffer (L2) is allocated in banks, while the inter-buffer (L1) is assigned variable lengths, requiring a balance between resource usage and bandwidth bottlenecks. The global buffer (L0) acts as an intermediary for subgraphs, with its space partitioned based on the size of the final output of each subgraph. Following this, memory spaces at each level are matched according to the data dependencies between nodes in the subgraph, using a "proximity to computation units first" principle, ultimately generating a complete data flow.

c) Evaluation Iteration: An evolutionary algorithm is employed to search for the optimal solution within the solution space. The objective function is defined to minimize latency of data move:

$$T_{DT} = \sum_{C_{sg}} T_{DSG} + \max_{k}(T_{DCT_k}) \tag{3}$$

$$T_{DCT} = 2 * (\sum_{layer} T_{ex}^{l'} + \sum_{layer} T_{in}^{l}) \tag{4}$$

$$T_{DSG} = \frac{F_{out_{ln}}}{Band_{L_0}}, T_{ex}^{l'} = \frac{Data^{l'}}{Band_{L_1}}, T_{in}^{l} = \frac{Data^{l}}{Band_{L_2}} \tag{5}$$

The total data transfer latency T_{DT} is composed of the subgraph data latency T_{DSG} and the maximum core task data latency T_{DCT}. C_{sg} represents the number of subgraphs, and $F_{out_{ln}}$ denotes the output feature map size of the last node in each subgraph. The T_{DCT} is further divided into intra-core latency T_{in} and inter-core latency T_{ex}, which are computed as the corresponding data volumes divided by the bandwidth of the respective level buffer.

In the mutation phase, adjustments are made by altering the ownership of the endpoint nodes of subgraphs and alternating the computational units of the execution nodes. This method allows the system to explore various configurations, ultimately identifying the optimal solution that minimizes latency. Typically, this results in a data flow with very few cross points, as indicated by the red lines in Fig. 4.

B. Weight Mapping

Due to the varying scales of crossbar arrays and the weight parameters of different network layers, the utilization of crossbars significantly impacts overall computational efficiency. It is crucial to develop an optimal method to automatically allocate the entire network's parameters to the arrays, thereby facilitating the complete model deployment in CIM systems. Additionally, since each storage cell within the crossbar also functions as a computation unit, weight replication is a vital approach to enhancing computational parallelism and crossbar array utilization.

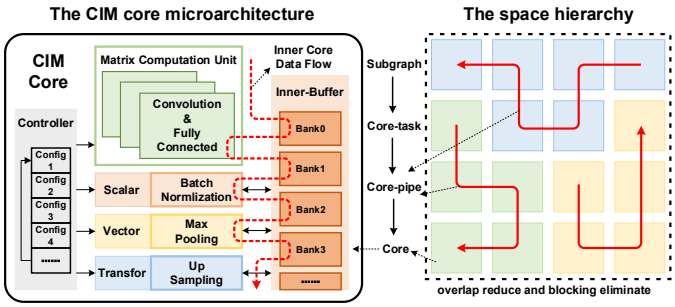

Fig. 4. Ideal smooth data flow lines within and between cores.

a) Integer Linear Programming: As shown in Fig. 6(a-c), mapping an 8-layer network onto a limited space. Different colored blocks represent the weight parameters of various MVM layers. In traditional sequential mapping methods, weight blocks are mapped to the crossbar array in the order they appear in the computation graph. When a block cannot fit into the available space, a new space is required, resulting in significant unused space. To address this issue, an Integer Linear Programming (ILP) space mapping strategy [13] has been proposed as an effective method to optimize crossbar utilization. This approach treats the boundary constraints of weight arrangement as an integer constraint problem, with the position of weight blocks as the ILP's objective. Through linear programming, a candidate set of all feasible solutions is constructed, and the optimal solution is selected.

b) Weight Replication: Replication strategies vary across different hierarchical levels. At the macro level, weight replication blocks are placed across different cores, directly enhancing parallelism by leveraging asynchronous inter-core operations. However, on a single crossbar array, to compute two inputs within the same cycle, it is necessary to concatenate the inputs. Due to the computational characteristics of crossbars, the fundamental approach involves diagonally separating the replicated weight blocks from the original ones to prevent computational interference. This approach necessitates significant zero-padding, as illustrated in Fig. 5(b). Given the sliding window nature of convolution operations, there is data reuse between adjacent input frames. Consequently, by merging

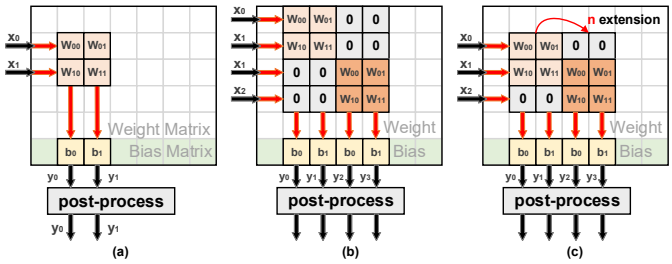

Fig. 5. (a) The original computation in crossbar array. (b) Weight replication for parallel computing and (c) OMM-like method to reduce zero padding.

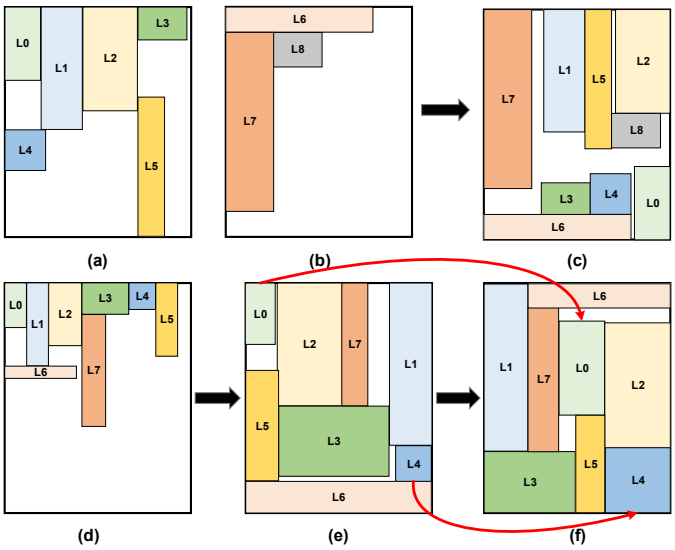

Fig. 6. Traditional sequential mapping methods (a)-(b), and ILP methods to improve the use ratio of crossbar. (d)-(e) ILP + OMM-like method for mapping and weighted skew on the bottleneck layer (L0 and L4).

input data, the zero-padding introduced by weight replication can be minimized. This method is similar in concept to the overlapped mapping method (OMM) [14].

Clearly, within a single large core, the OMM-like can help the system achieve an equivalent trade-off between crossbar area and computational efficiency. When increasing the output by M times, the crossbar utilization η (example for horizontal direction) can be determined using (6) and (7), where S_{array} denotes the area of the crossbar array.

$$T'_{MVM} = \frac{F_{out}}{M * C_{out}} \tag{6}$$

$$\eta = \frac{M * C_{out} * [K_w * K_h + (M-1)K_w]}{S_{array}} \tag{7}$$

Therefore, we have refined the original ILP by incorporating the horizontal and vertical expansion of each layer's weight parameters as new variables in the optimization process. Additionally, maximizing utilization has been introduced as one of the primary objectives. This enhancement has led to an increase in large core utilization to over 90%, and the substantial reduction in latency is directly proportional to the degree of weight expansion.

c) Weighted extension: In convolutional computations, the execution efficiency of different layers varies, often related to their sliding window operations. To mitigate the bottlenecks caused by excessive sliding window operations, we introduced a weighted scoring mechanism on top of the ILP + OMM-like approach. The reward function is designed as follows:

$$N_{w/h} = \left\lfloor \frac{IF_{w/h} - K_{w/h} + 2P_{w/h}}{S_{w/h}} \right\rfloor + 1 \tag{8}$$

$$N = N_w * N_h \tag{9}$$

$$F_{\text{Rewards}} = \frac{N}{M} x^{\frac{1}{\alpha}}, \quad \alpha > 1 \tag{10}$$

Where: N is the number of sliding window operations for the current layer. $IF_{w/h}$ represents the width and height of the input feature map. K, P, and S denote the kernel, padding, and stride, respectively. M is a normalization adjustment parameter, and F_{Rewards} is a fitted exponential function, where x is the extension factor based on the original weights. In the process of weighted extension solving in Fig. 6(d-f), array resources are prioritized for allocation to the bottleneck layers, resulting in further reduction of latency.

V. EVALUATION

A. Experiment Setup

a) Architecture Configuration: The final evaluation experiments were conducted directly on our SoC, which employs an eFlash floating-gate crossbar scheme. This SoC features a three-level buffer and consists of four large cores interconnected via a NoC. Each core contains eight MCUs and five OCUs, with MCU and OCUs executing in parallel, though only one MCU can execute at a time. The MCUs are organized in pairs to form a pipelined ping-pong structure, achieving one MVM every 120ns. The eFlash cell's precision is 8-bit, with inputs, outputs, and weights of crossbar all represented as 8-bit fixed-point numbers. Detailed configurations are in Table I.

TABLE I
METRICS OF OUR MULTI-CORE EFLASH-BASED CIM SOC

Component	***Params***	***Specific***	***Params***	***Specific***
Chip	core array	4	-	-
Global buffer	size	4MB	bandwidth	8GB/s
Core	MCU	2X4	OCU	5
	DAC	1152	ADC	320
Inter buffer	size	512KB	bandwidth	32GB/s
Inner buffer	bank num	8	bandwidth	64GB/s
	bank size	32KB		
Crossbar	weight size	1152*960	bias size	32*960
latency	MCU	240ns	OCU	4ns

b) benchmarks and baselines: We used commonly employed networks in computer visual scenarios (Yolov5, Gsr, Gvfe, Resnet50 and Unet) as benchmarks. Linear scheduling (sequential allocation in order) and OMM-like mapping strategies were selected as the baselines for our scheduling and mapping comparisons. We obtained latency measurements through direct testing on the SoC. Additionally, we developed a performance analysis simulator capable of event-level simulation in multi-level architectures, allowing us to analyze other performance metrics. These include different levels of buffer usage, computational unit utilization, and the time consumption of specific components.

B. Experiment Result

a) Resource Utilization: Fig. 8(left) illustrates the reduction in the number of chip pipelines for network tasks with varying complexities. This reduction indicates that more operators are processed within a single chip pipeline, leading to a maximum increase in computational resource utilization by up to 1.4 times. Additionally, storage resource utilization improved by up to 30%. Fig. 8(right) shows the variation

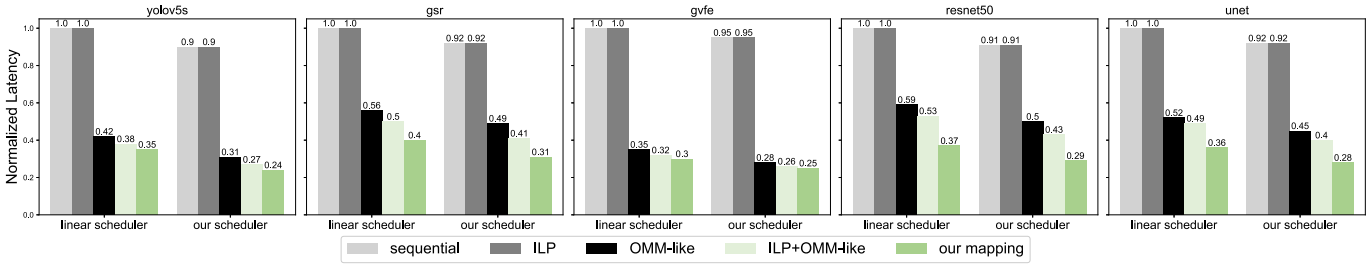

Fig. 7. The latency improvement with different strategy on benchmarks.

in data transfer latency between different levels of buffers. Although the latency of L1 increased, the higher bandwidth of L1 compared to L0 resulted in an overall 8% reduction in total latency.

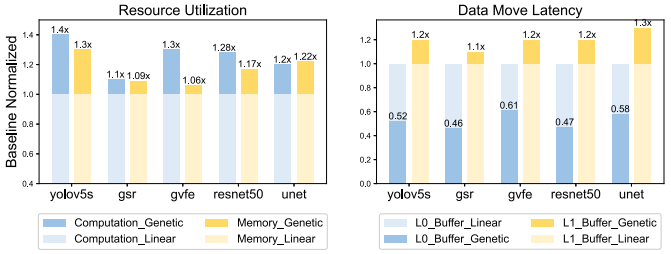

Fig. 8. The comparison of resource utilization for linear-method and our scheduler, and the latency change in different level of buffer.

b) Crossbar Utilization: Fig. 9 displays the improvements in area utilization and computational efficiency of crossbar arrays by different mapping strategies. In the case of large cores, ILP does not optimize performance without weight replication. However, with weight replication, there is a noticeable increase in area utilization, peaking at 94.7%. Although the overall area utilization decreases slightly after applying weighted skewing resources to address bottleneck layers, the latency is further improved. The final scheme achieves an average latency improvement of 28.2% compared to OMM-like strategies.

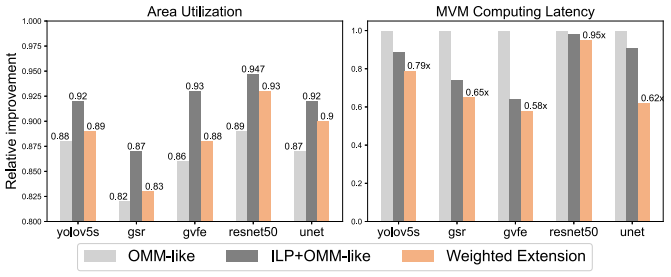

Fig. 9. Crossbar use ratio and computing latency of different mapping strategy.

c) Latency: Fig. 7 shows the latency performance of benchmark deployments on our SoC, corresponding to the combination of all optimization strategies. The data indicates that our resource scheduling and weight mapping strategies independently contribute to a maximum latency reduction of 10% and 70%, respectively. For the combined deployment on the Yolov5s network, the overall latency was reduced by 76%.

VI. Conclusion

This work pioneers the automatic optimization and deployment of multi-core, large-array CIM SoCs. Previous research has often overlooked domain-specific compilers in multi-core, large-array systems, with some efforts still confined to simulation stages. We addressed the growing demand for low-latency edge scenarios by developing optimization strategies for resource scheduling and weight mapping. Experimental results demonstrate a 30% improvement in resource utilization, a 94.7% crossbar array utilization, and a 76% reduction in latency based on actual chip measurements.

References

[1] K. H. Lee and et al., "A Low-Power Processor With Configurable Embedded Machine-Learning Accelerators for High-Order and Adaptive Analysis of Medical-Sensor Signals," JSSC, vol. 48, no. 7, pp. 1625-1637, 2013.
[2] T. Chen and et al., "DianNao: a small-footprint high-throughput accelerator for ubiquitous machine-learning," in ASPLOS, pp. 269–284, 2014.
[3] K. Lee and et al., "A Charge-Sharing based 8T SRAM In-Memory Computing for Edge DNN Acceleration," DAC, pp. 739-744, 2021.
[4] C. Wolters and et al., "Memory Is All You Need: An Overview of Compute-in-Memory Architectures for Accelerating Large Language Model Inference," arXiv: 2406.08413 [cs.AR].
[5] R. Pelke and et al., "Mapping of CNNs on multi-core RRAM-based CIM architectures," VLSI-SoC, pp. 1-6, 2023.
[6] K. E. Jeon and et al., "Weight-Aware Activation Mapping for Energy-Efficient Convolution on PIM Arrays," ISLPED, pp. 1-6, 2023.
[7] J. Bai and et al., "CIMQ: A Hardware-Efficient Quantization Framework for Computing-In-Memory-Based Neural Network Accelerators," TCAD, vol. 43, no. 1, pp. 189-202, 2023.
[8] X. Guo and et al., "Fast, energy-efficient, robust, and reproducible mixed-signal neuromorphic classifier based on embedded NOR flash memory technology," IEDM, pp. 6.5.1-6.5.4, 2017.
[9] Y. Feng and et al., "A Novel Array Programming Scheme for Large Matrix Processing in Flash-Based Computing-in-Memory (CIM) With Ultrahigh Bit Density," TED, vol. 70, no. 2, pp. 461-467, 2023.
[10] C. Yang and et al., "CIMAX-Compiler: An End-to-End ANN Compiler for Heterogeneous Computing-in-Memory Platform," WCCCT, pp. 152-157, 2023.
[11] X. Sun and et al., "PIMCOMP: A Universal Compilation Framework for Crossbar-based PIM DNN Accelerators," DAC, pp. 1-6, 2023.
[12] L. Han and et al., "A Convolution Neural Network Accelerator Design with Weight Mapping and Pipeline Optimization," DAC, pp. 1-6, 2023.
[13] T. Bai and et al., "An End-to-End In-Memory Computing System Based on a 40-nm eFlash-Based IMC SoC: Circuits, Toolchains, and Systems Co-Design Framework," TCAD, vol. 43, no. 6, pp. 1729-1740, 2024.
[14] Z. Zhu and et al., "Mixed Size Crossbar based RRAM CNN Accelerator with Overlapped Mapping Method," ICCAD. pp, 1–8, 2018.